

Week 2 Assignment Report

Objective

The objective of this assignment is to develop practical skills in NumPy and Pandas, which are essential Python libraries for Data Analytics. Students will learn to work with arrays, perform mathematical and statistical operations, manipulate datasets, inspect and clean data, filter and sort records, combine datasets, perform GroupBy and Pivot Table operations, and export processed data.

Task 1: NumPy Introduction & Arrays

Create a Python program using NumPy that: Imports NumPy. Creates a NumPy array containing at least 10 numbers. Displays the array. Displays the array's shape, size, and data type. Creates a one-dimensional and two-dimensional array.

-----code-----

```
import numpy as np

# Create a NumPy array containing 10 numbers
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

# Display the array
print("NumPy Array:")
print(arr)

# Display shape, size, and data type
print("\nShape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)

# Create a one-dimensional array
one_d = np.array([1, 2, 3, 4, 5])
print("\nOne-Dimensional Array:")
print(one_d)

# Create a two-dimensional array
two_d = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print("\nTwo-Dimensional Array:")  
print(two_d)
```

-----output-----

```
NumPy Array:  
[ 10  20  30  40  50  60  70  80  90 100]  
  
Shape: (10,)  
Size: 10  
Data Type: int32  
  
One-Dimensional Array:  
[1 2 3 4 5]  
  
Two-Dimensional Array:  
[[1 2 3]  
 [4 5 6]]
```

Task 2: NumPy Indexing, Slicing & Reshaping

Using a NumPy array: Access specific elements using indexing. Extract a portion of the array using slicing. Create a two-dimensional array. Access specific rows and columns. Reshape an array into different dimensions. Display the original and reshaped arrays.

-----code-----

```
import numpy as np  
  
# Create a NumPy array  
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])  
print("Original Array:")  
print(arr)  
  
# Access specific elements using indexing  
print("\nElement at index 2:", arr[2])  
print("Element at index 5:", arr[5])  
  
# Extract a portion of the array using slicing
```

```
print("\nSliced Array:")
print(arr[2:7])

# Create a two-dimensional array
two_d = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print("\nTwo-Dimensional Array:")
print(two_d)

# Access specific rows
print("\nFirst Row:")
print(two_d[0])
print("\nSecond Row:")
print(two_d[1])

# Access specific columns
print("\nFirst Column:")
print(two_d[:, 0])
print("\nSecond Column:")
print(two_d[:, 1])

# Reshape the original array
reshaped = arr.reshape(2, 5)

# Display original and reshaped arrays
print("\nOriginal Array:")
print(arr)
print("\nReshaped Array (2 x 5):")
print(reshaped)
```

-----output-----

Original Array:

```
[ 10  20  30  40  50  60  70  80  90 100]
```

Element at index 2: 30

Element at index 5: 60

Sliced Array:

```
[30 40 50 60 70]
```

Two-Dimensional Array:

```
[[1 2 3]
```

```
 [4 5 6]
```

```
 [7 8 9]]
```

First Row:

```
[1 2 3]
```

Second Row:

```
[4 5 6]
```

First Column:

```
[1 4 7]
```

Second Column:

```
[2 5 8]
```

Original Array:

```
[ 10  20  30  40  50  60  70  80  90 100]
```

Reshaped Array (2 x 5):

```
[[ 10  20  30  40  50]
```

```
 [ 60  70  80  90 100]]
```

Task 3: NumPy Mathematical & Statistical Operations

Perform the following operations using NumPy: Mathematical Operations Addition Subtraction Multiplication Division Statistical Functions Mean Median Minimum Maximum Standard Deviation Sum Use a numerical dataset of your choice and display the results.

-----code-----

```
import numpy as np

# Create two NumPy arrays
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([2, 4, 5, 8, 10])

print("Array 1:")
print(arr1)

print("\nArray 2:")
print(arr2)

# Mathematical Operations
print("\n--- Mathematical Operations ---")

print("Addition:")
print(arr1 + arr2)

print("Subtraction:")
print(arr1 - arr2)

print("Multiplication:")
print(arr1 * arr2)

print("Division:")
print(arr1 / arr2)

# Statistical Operations
print("\n--- Statistical Operations ---")

print("Mean:", np.mean(arr1))
print("Median:", np.median(arr1))
print("Minimum:", np.min(arr1))
print("Maximum:", np.max(arr1))
```

```
print("Standard Deviation:", np.std(arr1))
```

```
print("Sum:", np.sum(arr1))
```

```
# Take two numbers as input
```

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
# Perform operations
```

```
print("\n----- Calculator Results -----")
```

```
print("Addition      :", num1 + num2)
```

```
print("Subtraction   :", num1 - num2)
```

```
print("Multiplication :", num1 * num2)
```

```
print("Division      :", num1 / num2)
```

```
print("Modulus       :", num1 % num2)
```

-----output-----

```
Array 1:
[10 20 30 40 50]

Array 2:
[ 2  4  5  8 10]

--- Mathematical Operations ---
Addition:
[12 24 35 48 60]
Subtraction:
[ 8 16 25 32 40]
Multiplication:
[ 20  80 150 320 500]
Division:
[5. 5. 6. 5. 5.]

--- Statistical Operations ---
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard Deviation: 14.142135623730951
Sum: 150
```

Task 4: Pandas Series & DataFrame

Create a Pandas program that: Creates a Pandas Series. Creates a DataFrame containing student or employee information. Displays the DataFrame. Displays the column names. Displays the index. Adds a new column to the DataFrame. Displays the updated DataFrame.

-----code-----

```
import pandas as pd

# Create a Pandas Series

marks = pd.Series([85, 92, 78, 88, 95])

print("Pandas Series:")

print(marks)
```

```
# Create a DataFrame containing student information

data = {
    "Name": ["Aarav", "Riya", "Karan", "Meera", "Priya"],
    "Age": [20, 21, 20, 22, 20],
    "Marks": [85, 92, 78, 88, 95]
}

df = pd.DataFrame(data)

# Display the DataFrame
print("\nStudent DataFrame:")

print(df)

# Display column names
print("\nColumn Names:")

print(df.columns)

# Display the index
print("\nIndex:")

print(df.index)

# Add a new column
df["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]

# Display the updated DataFrame
print("\nUpdated DataFrame:")

print(df)
```

-----**output**-----

Pandas Series:

```
0    85
1    92
2    78
3    88
4    95
```

dtype: int64

Student DataFrame:

	Name	Age	Marks
0	Aarav	20	85
1	Riya	21	92
2	Karan	20	78
3	Meera	22	88
4	Priya	20	95

Column Names:

Index(['Name', 'Age', 'Marks'], dtype='object')

Index:

RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:

	Name	Age	Marks	Result
0	Aarav	20	85	Pass
1	Riya	21	92	Pass
2	Karan	20	78	Pass
3	Meera	22	88	Pass
4	Priya	20	95	Pass

Task 5: Reading & Inspecting Data

Use a CSV dataset and perform the following operations: Read the CSV file using Pandas. Display the first 5 rows using head(). Display the last 5 rows using tail(). Check the number of rows and columns. Display column names. Check data types using dtypes. Use info() and describe() to understand the dataset.

-----code-----

```
import pandas as pd
```

```
import seaborn as sns

# Load the Titanic dataset using Seaborn
df = sns.load_dataset("titanic")

# Display the first 5 rows
print("First 5 Rows:")
print(df.head())

# Display the last 5 rows
print("\nLast 5 Rows:")
print(df.tail())

# Check the number of rows and columns
print("\nNumber of Rows and Columns:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Check data types
print("\nData Types:")
print(df.dtypes)

# Display information about the dataset
print("\nDataset Information:")
df.info()
```

```
# Display statistical summary

print("\nStatistical Description:")

print(df.describe())
```

-----output-----

First 5 Rows:										
	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	
	who	adult_male	deck	embark_town	alive	alone				
0	man	True	NaN	Southampton	no	False				
1	woman	False	C	Cherbourg	yes	False				
2	woman	False	NaN	Southampton	yes	True				
3	woman	False	C	Southampton	yes	False				
4	man	True	NaN	Southampton	no	True				
Last 5 Rows:										
	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
886	0	2	male	27.0	0	0	13.00	S	Second	
887	1	1	female	19.0	0	0	30.00	S	First	
888	0	3	female	NaN	1	2	23.45	S	Third	
889	1	1	male	26.0	0	0	30.00	C	First	
890	0	3	male	32.0	0	0	7.75	Q	Third	
	who	adult_male	deck	embark_town	alive	alone				
886	man	True	NaN	Southampton	no	True				
887	woman	False	B	Southampton	yes	True				
888	woman	False	NaN	Southampton	no	False				
889	man	True	C	Cherbourg	yes	True				
890	man	True	NaN	Queenstown	no	True				

```
Number of Rows and Columns:  
(891, 15)
```

```
Column Names:
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
      'alive', 'alone'],  
      dtype='object')
```

```
Data Types:
```

```
survived      int64  
pclass        int64  
sex           object  
age           float64  
sibsp         int64  
parch         int64  
fare          float64  
embarked      object  
class         category  
who           object  
adult_male    bool  
deck          category  
embark_town   object  
alive         object  
alone         bool  
dtype: object
```

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male   891 non-null    bool
11  deck         203 non-null    category
12  embark_town  889 non-null    object
13  alive        891 non-null    object
14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

Statistical Description:

```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000

```

Statistical Description:

```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Task 6: Selecting, Filtering & Sorting Data

Using a Pandas DataFrame: Select specific columns. Select specific rows. Filter records based on a condition. Apply multiple filtering conditions. Sort data in ascending order. Sort data in descending order.

-----code-----

```

import pandas as pd

import seaborn as sns

# Load Titanic dataset

df = sns.load_dataset("titanic")

```

```
# Select specific columns
print("Selected Columns:")
print(df[["survived", "sex", "age", "fare"]].head())

# Select specific rows
print("\nSelected Rows:")
print(df.iloc[0:5])

# Filter passengers based on a condition
print("\nPassengers with Fare Greater Than 100:")
print(df[df["fare"] > 100][["sex", "age", "fare", "survived"]])

# Multiple filtering conditions
print("\nFemale Passengers Above 30 Years:")
print(
    df[
        (df["sex"] == "female") &
        (df["age"] > 30)
    ][["sex", "age", "fare", "survived"]]
)

# Sort in ascending order
print("\nPassengers Sorted by Age - Ascending:")
print(df.sort_values(by="age", ascending=True)[["sex", "age", "fare"]].head(10))

# Sort in descending order
print("\nPassengers Sorted by Fare - Descending:")
print(df.sort_values(by="fare", ascending=False)[["sex", "age", "fare"]].head(10))
```

-----output-----

Selected Columns:

	survived	sex	age	fare
0	0	male	22.0	7.2500
1	1	female	38.0	71.2833
2	1	female	26.0	7.9250
3	1	female	35.0	53.1000
4	0	male	35.0	8.0500

Selected Rows:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Passengers with Fare Greater Than 100:

	sex	age	fare	survived
...				
438	male	64.0	263.0000	
311	female	18.0	262.3750	
742	female	21.0	262.3750	
118	male	24.0	247.5208	

Female Passengers Above 30 Years:

	sex	age	fare	survived
1	female	38.0	71.2833	1
3	female	35.0	53.1000	1
11	female	58.0	26.5500	1
15	female	55.0	16.0000	1
18	female	31.0	18.0000	0
..	...	...	...	...
862	female	48.0	25.9292	1
865	female	42.0	13.0000	1
871	female	47.0	52.5542	1
879	female	56.0	83.1583	1
885	female	39.0	29.1250	0

Passengers Sorted by Age - Ascending:

	sex	age	fare
803	male	0.42	8.5167
755	male	0.67	14.5000
644	female	0.75	19.2583
469	female	0.75	19.2583
78	male	0.83	29.0000
831	male	0.83	18.7500
305	male	0.92	151.5500
827	male	1.00	37.0042
381	female	1.00	15.7417
164	male	1.00	39.6875

Passengers Sorted by Fare - Descending:

	sex	age	fare
258	female	35.0	512.3292
737	male	35.0	512.3292
679	male	36.0	512.3292
88	female	23.0	263.0000
27	male	19.0	263.0000
341	female	24.0	263.0000
438	male	64.0	263.0000
311	female	18.0	262.3750
742	female	21.0	262.3750
118	male	24.0	247.5208

Passengers Sorted by Fare - Descending:

	sex	age	fare
258	female	35.0	512.3292
737	male	35.0	512.3292
679	male	36.0	512.3292
88	female	23.0	263.0000
27	male	19.0	263.0000
341	female	24.0	263.0000
438	male	64.0	263.0000
311	female	18.0	262.3750
742	female	21.0	262.3750
118	male	24.0	247.5208

Task 7: Handling Missing Values

Take a dataset containing missing values and perform the following: Identify missing values using `isnull()` / `isna()`. Count missing values in each column. Remove rows containing missing values. Fill missing values using appropriate methods. Display the dataset before and after handling missing values. Briefly explain why handling missing data is important in Data Analytics.

-----code-----

```
import pandas as pd
import seaborn as sns

# Load Titanic dataset
df = sns.load_dataset("titanic")

# Display dataset before handling missing values
print("Dataset Before Handling Missing Values:")
print(df.head(10))

# Identify missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Remove rows containing missing values
df_dropped = df.dropna()
print("\nDataset After Removing Rows With Missing Values:")
print(df_dropped.head())

# Create a copy for filling missing values
df_filled = df.copy()

# Fill missing Age with mean
df_filled["age"] = df_filled["age"].fillna(
    df_filled["age"].mean()
)

# Fill missing Embarked with mode
df_filled["embarked"] = df_filled["embarked"].fillna(
    df_filled["embarked"].mode()[0]
)

# Fill missing Embark Town with mode
df_filled["embark_town"] = df_filled["embark_town"].fillna(
```

```
df_filled["embark_town"].mode()[0]
)
# Add "Unknown" as a category before filling missing Deck values
df_filled["deck"] = df_filled["deck"].cat.add_categories("Unknown")
df_filled["deck"] = df_filled["deck"].fillna("Unknown")
# Display dataset after filling missing values
print("\nDataset After Filling Missing Values:")
print(df_filled.head(10))
# Check missing values after handling
print("\nMissing Values After Handling:")
print(df_filled.isnull().sum())
```

-----**output**-----

Dataset Before Handling Missing Values:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third
5	0	3	male	NaN	0	0	8.4583	Q	Third
6	0	1	male	54.0	0	0	51.8625	S	First
7	0	3	male	2.0	3	1	21.0750	S	Third
8	1	3	female	27.0	0	2	11.1333	S	Third
9	1	2	female	14.0	1	0	30.0708	C	Second

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True
5	man	True	NaN	Queenstown	no	True
6	man	True	E	Southampton	no	True
7	child	False	NaN	Southampton	no	False
8	woman	False	NaN	Southampton	yes	False
9	child	False	NaN	Cherbourg	yes	False

Missing Values:

survived	0
pclass	0
sex	0
age	177

Dataset After Removing Rows With Missing Values:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
1	1	1	female	38.0	1	0	71.2833	C	First	
3	1	1	female	35.0	1	0	53.1000	S	First	
6	0	1	male	54.0	0	0	51.8625	S	First	
10	1	3	female	4.0	1	1	16.7000	S	Third	
11	1	1	female	58.0	0	0	26.5500	S	First	

	who	adult_male	deck	embark_town	alive	alone
1	woman	False	C	Cherbourg	yes	False
3	woman	False	C	Southampton	yes	False
6	man	True	E	Southampton	no	True
10	child	False	G	Southampton	yes	False
11	woman	False	C	Southampton	yes	True

Dataset After Filling Missing Values:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	\
0	0	3	male	22.000000	1	0	7.2500	S	
1	1	1	female	38.000000	1	0	71.2833	C	
2	1	3	female	26.000000	0	0	7.9250	S	
3	1	1	female	35.000000	1	0	53.1000	S	
4	0	3	male	35.000000	0	0	8.0500	S	
5	0	3	male	29.699118	0	0	8.4583	Q	
6	0	1	male	54.000000	0	0	51.8625	S	
7	0	3	male	2.000000	3	1	21.0750	S	
8	1	3	female	27.000000	0	2	11.1333	S	
9	1	2	female	14.000000	1	0	30.0708	C	

class who adult_male deck embark_town alive alone

	class	who	adult_male	deck	embark_town	alive	alone
0	Third	man	True	Unknown	Southampton	no	False
1	First	woman	False	C	Cherbourg	yes	False
2	Third	woman	False	Unknown	Southampton	yes	True
3	First	woman	False	C	Southampton	yes	False
4	Third	man	True	Unknown	Southampton	no	True
5	Third	man	True	Unknown	Queenstown	no	True
6	First	man	True	E	Southampton	no	True
7	Third	child	False	Unknown	Southampton	no	False
8	Third	woman	False	Unknown	Southampton	yes	False
9	Second	child	False	Unknown	Cherbourg	yes	False

Missing Values After Handling:

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck          0
embark_town   0
alive         0
alone         0
dtype: int64
```

Task 8: Merge, Concatenate, GroupBy & Pivot Table

Using two or more datasets, perform the following operations: Merge & Concatenate Merge two DataFrames using a common column. Concatenate two DataFrames. GroupBy Group data based on a selected column. Calculate aggregate values such as sum, mean, count, minimum, or maximum. Pivot Table Create a Pivot Table using Pandas. Summarize the data based on appropriate rows, columns, and values. Display the output of each operation.

-----code-----

```
import pandas as pd

import seaborn as sns

# Load Titanic dataset

df = sns.load_dataset("titanic")

# =====

# 1. CREATE TWO DATAFRAMES

# =====

df1 = df[[

    "survived",

    "pclass",

    "sex",

    "age"

]].head(10).copy()

df1["Passenger_ID"] = range(1, 11)

df2 = df[[

    "fare",

    "embarked",

    "class"

]].head(10).copy()

df2["Passenger_ID"] = range(1, 11)

print("DataFrame 1:")

print(df1)

print("\nDataFrame 2:")
```

```

print(df2)

# =====

# 2. MERGE

# =====

merged_df = pd.merge(
    df1,
    df2,
    on="Passenger_ID"
)

print("\n===== MERGED DATAFRAME =====")

print(merged_df)

# =====

# 3. CONCATENATE

# =====

part1 = df.head(5)
part2 = df.iloc[5:10]
concatenated_df = pd.concat(
    [part1, part2],
    ignore_index=True
)

print("\n===== CONCATENATED DATAFRAME =====")

print(concatenated_df)

# =====

# 4. GROUPBY

# =====

print("\n===== GROUPBY: SURVIVAL BY GENDER =====")

grouped = df.groupby("sex")["survived"].mean()

print(grouped)

```

```

# =====
# 5. AGGREGATE FUNCTIONS
# =====
print("\n===== AGGREGATE ANALYSIS =====")
aggregate = df.groupby("class")["fare"].agg(
    ["sum", "mean", "count", "min", "max"]
)
print(aggregate)
# =====
# 6. PIVOT TABLE
# =====
print("\n===== PIVOT TABLE =====")

pivot = pd.pivot_table(
    df,
    values="survived",
    index="class",
    columns="sex",
    aggfunc="mean"
)
print(pivot)

```

-----output-----

Task 9: Mini Python Project

Develop a Student Record Management System using Python. The program should: Add student details. Display all student records. Search a student by name. Delete a student record. Use lists or dictionaries for data storage.

code

```
DataFrame 1:
  survived  pclass    sex  age  Passenger_ID
0         0      3   male 22.0           1
1         1      1  female 38.0           2
2         1      3  female 26.0           3
3         1      1  female 35.0           4
4         0      3   male 35.0           5
5         0      3   male  NaN           6
6         0      1   male 54.0           7
7         0      3   male  2.0           8
8         1      3  female 27.0           9
9         1      2  female 14.0          10

DataFrame 2:
   fare embarked  class  Passenger_ID
0  7.2500      S   Third           1
1 71.2833      C   First           2
2  7.9250      S   Third           3
3 53.1000      S   First           4
4  8.0500      S   Third           5
5  8.4583      Q   Third           6
6 51.8625      S   First           7
7 21.0750      S   Third           8
8 11.1333      S   Third           9
9 30.0708      C  Second          10

===== MERGED DATAFRAME =====
  survived  pclass    sex  age  Passenger_ID  fare embarked  class
0         0      3   male 22.0           1  7.2500      S   Third
1         1      1  female 38.0           2 71.2833      C   First
```

```
===== MERGED DATAFRAME =====
  survived  pclass    sex  age  Passenger_ID  fare embarked  class
0         0      3   male 22.0           1  7.2500      S   Third
1         1      1  female 38.0           2 71.2833      C   First
2         1      3  female 26.0           3  7.9250      S   Third
3         1      1  female 35.0           4 53.1000      S   First
4         0      3   male 35.0           5  8.0500      S   Third
5         0      3   male  NaN           6  8.4583      Q   Third
6         0      1   male 54.0           7 51.8625      S   First
7         0      3   male  2.0           8 21.0750      S   Third
8         1      3  female 27.0           9 11.1333      S   Third
9         1      2  female 14.0          10 30.0708      C  Second

===== CONCATENATED DATAFRAME =====
  survived  pclass    sex  age  sibsp  parch  fare embarked  class \
0         0      3   male 22.0      1      0  7.2500      S   Third
1         1      1  female 38.0      1      0 71.2833      C   First
2         1      3  female 26.0      0      0  7.9250      S   Third
3         1      1  female 35.0      1      0 53.1000      S   First
4         0      3   male 35.0      0      0  8.0500      S   Third
5         0      3   male  NaN      0      0  8.4583      Q   Third
6         0      1   male 54.0      0      0 51.8625      S   First
7         0      3   male  2.0      3      1 21.0750      S   Third
8         1      3  female 27.0      0      2 11.1333      S   Third
9         1      2  female 14.0      1      0 30.0708      C  Second

  who  adult_male  deck  embark_town  alive  alone
0  man         True  NaN  Southampton    no  False
1 woman        False   C   Cherbourg   yes  False
2 woman        False  NaN  Southampton   yes   True
```

```

   who  adult_male  deck  embark_town  alive  alone
0   man         True  NaN  Southampton    no  False
1  woman         False   C   Cherbourg   yes  False
2  woman         False  NaN  Southampton   yes   True
3  woman         False   C   Southampton   yes  False
4   man         True  NaN  Southampton    no   True
5   man         True  NaN  Queenstown    no   True
6   man         True   E   Southampton    no   True
7  child         False  NaN  Southampton    no  False
8  woman         False  NaN  Southampton   yes  False
9  child         False  NaN   Cherbourg   yes  False

```

===== GROUPBY: SURVIVAL BY GENDER =====

```

sex
female    0.742038
male      0.188908
Name: survived, dtype: float64

```

===== AGGREGATE ANALYSIS =====

```

              sum      mean  count  min      max
class
First  18177.4125  84.154687    216  0.0  512.3292
Second  3801.8417  20.662183    184  0.0   73.5000
Third   6714.6951  13.675550    491  0.0   69.5500

```

===== PIVOT TABLE =====

```

sex      female      male
class
First    0.968085    0.368852

```

Task 9: Exporting Data (5 Marks) After processing the dataset: Export the final DataFrame to a new CSV file. Verify that the exported file contains the processed data. Save the file properly for submission.

-----code-----

```

import pandas as pd
import seaborn as sns

# Load Titanic dataset
df = sns.load_dataset("titanic")

print("Original Dataset:")
print(df.head())

# Handle missing Age
df["age"] = df["age"].fillna(
    df["age"].mean()
)

# Handle missing Embarked
df["embarked"] = df["embarked"].fillna(

```

```
    df["embarked"].mode()[0]
)
# Handle missing Embark Town
df["embark_town"] = df["embark_town"].fillna(
    df["embark_town"].mode()[0]
)
# Handle missing Deck
df["deck"] = df["deck"].cat.add_categories("Unknown")
df["deck"] = df["deck"].fillna("Unknown")
print("\nProcessed Dataset:")
print(df.head())
# Export processed dataset
output_file = "cleaned_titanic.csv"
df.to_csv(output_file, index=False)
print("\nData exported successfully!")
print("File name:", output_file)
# Verify exported file
verified_df = pd.read_csv(output_file)
print("\n===== VERIFIED EXPORTED DATA =====")
print(verified_df.head())
print("\nExported Dataset Shape:")
print(verified_df.shape)
```

-----output-----

Original Dataset:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Processed Dataset:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	Unknown	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	Unknown	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	Unknown	Southampton	no	True

	who	adult_male	deck	embark_town	alive	alone
0	man	True	Unknown	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	Unknown	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	Unknown	Southampton	no	True

Data exported successfully!

File name: cleaned_titanic.csv

===== VERIFIED EXPORTED DATA =====

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	Unknown	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	Unknown	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	Unknown	Southampton	no	True

Exported Dataset Shape:

(891, 15)

-----code-----

Task 10: Mini Data Analysis Project

Perform a basic data analysis project using NumPy and Pandas. Choose a dataset such as:
Student Performance Dataset Sales Dataset Employee Dataset E-commerce Dataset
Customer Dataset

```
import pandas as pd

import numpy as np

import seaborn as sns

# =====

# 1. LOAD DATASET

# =====

df = sns.load_dataset("titanic")

print("===== TITANIC DATASET =====")

print(df.head())

# =====

# 2. DATA INSPECTION

# =====

print("\n===== FIRST 5 ROWS =====")

print(df.head())

print("\n===== LAST 5 ROWS =====")

print(df.tail())

print("\n===== DATASET SHAPE =====")

print(df.shape)

print("\n===== COLUMN NAMES =====")

print(df.columns)

print("\n===== DATA TYPES =====")

print(df.dtypes)

print("\n===== DATASET INFORMATION =====")

df.info()
```

```

print("\n===== STATISTICAL SUMMARY =====")
print(df.describe())

# =====

# 3. IDENTIFY MISSING VALUES

# =====

print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

# =====

# 4. HANDLE MISSING VALUES

# =====

# Fill missing Age with mean
df["age"] = df["age"].fillna(
    df["age"].mean()
)

# Fill missing Embarked with mode
df["embarked"] = df["embarked"].fillna(
    df["embarked"].mode()[0]
)

# Fill missing Embark Town with mode
df["embark_town"] = df["embark_town"].fillna(
    df["embark_town"].mode()[0]
)

# Convert deck to normal object column
# and fill missing values with "Unknown"
df["deck"] = df["deck"].astype("object").fillna("Unknown")

print("\n===== MISSING VALUES AFTER HANDLING =====")
print(df.isnull().sum())

# =====

```

5. SELECTING DATA

```
# =====  
print("\n===== SELECTED COLUMNS =====")
```

```
print(  
    df[  
        ["survived", "sex", "age", "fare"]  
    ].head(10)  
)
```

```
# =====
```

6. FILTERING DATA

```
# =====  
print("\n===== FEMALE PASSENGERS =====")
```

```
female_passengers = df[  
    df["sex"] == "female"  
]
```

```
print(  
    female_passengers[  
        ["sex", "age", "fare", "survived"]  
    ].head(10)  
)
```

```
print("\n===== PASSENGERS WITH FARE > 100 =====")
```

```
high_fare = df[  
    df["fare"] > 100
```

```

]
print(
    high_fare[
        ["sex", "age", "fare", "survived"]
    ].head(10)
)
# Multiple filtering conditions
print("\n===== FEMALE PASSENGERS WITH AGE > 30 =====")

filtered_data = df[
    (df["sex"] == "female") &
    (df["age"] > 30)
]
print(
    filtered_data[
        ["sex", "age", "fare", "survived"]
    ].head(10)
)
# =====

# 7. SORTING DATA
# =====

print("\n===== HIGHEST FARES =====")
sorted_df = df.sort_values(
    by="fare",
    ascending=False
)
print(
    sorted_df[

```

```

        ["sex", "age", "fare", "survived"]
    ].head(10)
)
print("\n===== LOWEST FARES =====")

sorted_low = df.sort_values(
    by="fare",
    ascending=True
)
print(
    sorted_low[
        ["sex", "age", "fare", "survived"]
    ].head(10)
)
# =====

# 8. GROUPBY ANALYSIS

# =====

print("\n===== SURVIVAL RATE BY GENDER =====")
gender_survival = df.groupby(
    "sex"
)["survived"].mean()

print(gender_survival)
print("\n===== SURVIVAL RATE BY CLASS =====")
class_survival = df.groupby(
    "class"
)["survived"].mean()

```

```

print(class_survival)

# =====

# 9. AGGREGATE ANALYSIS

# =====

print("\n===== FARE ANALYSIS BY CLASS =====")

fare_analysis = df.groupby(
    "class"
)["fare"].agg(
    ["sum", "mean", "min", "max", "count"]
)

print(fare_analysis)

# =====

# 10. PIVOT TABLE

# =====

print("\n===== PIVOT TABLE =====")

pivot = pd.pivot_table(
    df,
    values="survived",
    index="class",
    columns="sex",
    aggfunc="mean"
)

print(pivot)

# =====

# 11. NUMPY ANALYSIS

# =====

```

```

print("\n===== NUMPY ANALYSIS =====")

average_age = np.mean(df["age"])
average_fare = np.mean(df["fare"])
maximum_fare = np.max(df["fare"])
minimum_fare = np.min(df["fare"])

print("Average Age:", round(average_age, 2))
print("Average Fare:", round(average_fare, 2))
print("Maximum Fare:", maximum_fare)
print("Minimum Fare:", minimum_fare)

# =====

# 12. USEFUL INSIGHTS

# =====

print("\n===== USEFUL INSIGHTS =====")

# Overall survival rate
survival_rate = df["survived"].mean() * 100

print(
    "Overall Survival Rate:",
    round(survival_rate, 2),
    "%"
)

# Gender with highest survival rate
best_gender = gender_survival.idxmax()

print(
    "Gender with Higher Survival Rate:",
    best_gender
)

# Class with highest survival rate
best_class = class_survival.idxmax()

```

```

print(
    "Class with Highest Survival Rate:",
    best_class
)
# Average fare
print(
    "Average Ticket Fare:",
    round(df["fare"].mean(), 2)
)
# =====

# 13. EXPORT CLEANED DATASET
# =====

output_file = "cleaned_titanic.csv"
df.to_csv(
    output_file,
    index=False
)
print("\n===== EXPORT =====")
print(
    "Cleaned dataset saved as:",
    output_file
)
# =====

# 14. VERIFY EXPORTED DATA
# =====

verified_df = pd.read_csv(
    output_file
)

```

```

print("\n===== EXPORTED DATA =====")

print(verified_df.head())

print("\nExported Dataset Shape:")

print(verified_df.shape)

print("\nExported Dataset Columns:")

print(verified_df.columns)

```

-----output-----

```

===== TITANIC DATASET =====
  survived  pclass    sex  age  sibsp  parch    fare embarked  class \
0         0       3   male  22.0     1     0   7.2500         S   Third
1         1       1  female  38.0     1     0  71.2833         C   First
2         1       3  female  26.0     0     0   7.9250         S   Third
3         1       1  female  35.0     1     0  53.1000         S   First
4         0       3   male  35.0     0     0   8.0500         S   Third

   who  adult_male  deck  embark_town  alive  alone
0  man         True  NaN  Southampton    no  False
1 woman        False   C   Cherbourg   yes  False
2 woman        False  NaN  Southampton   yes   True
3 woman        False   C   Southampton   yes  False
4  man         True  NaN  Southampton    no   True

===== FIRST 5 ROWS =====
  survived  pclass    sex  age  sibsp  parch    fare embarked  class \
0         0       3   male  22.0     1     0   7.2500         S   Third
1         1       1  female  38.0     1     0  71.2833         C   First
2         1       3  female  26.0     0     0   7.9250         S   Third
3         1       1  female  35.0     1     0  53.1000         S   First
4         0       3   male  35.0     0     0   8.0500         S   Third

   who  adult_male  deck  embark_town  alive  alone
0  man         True  NaN  Southampton    no  False
1 woman        False   C   Cherbourg   yes  False
2 woman        False  NaN  Southampton   yes   True
3 woman        False   C   Southampton   yes  False
4  man         True  NaN  Southampton    no   True

```

```

===== LAST 5 ROWS =====
   survived  pclass    sex  age  sibsp  parch   fare embarked   class \
886         0      2  male  27.0     0     0  13.00         S  Second
887         1      1 female  19.0     0     0  30.00         S   First
888         0      3 female   NaN     1     2  23.45         S   Third
889         1      1  male  26.0     0     0  30.00         C   First
890         0      3  male  32.0     0     0   7.75         Q   Third

   who  adult_male deck  embark_town alive  alone
886  man         True  NaN  Southampton    no   True
887 woman        False   B  Southampton   yes   True
888 woman        False  NaN  Southampton    no  False
889  man         True   C   Cherbourg   yes   True
890  man         True  NaN  Queenstown    no   True

===== DATASET SHAPE =====
(891, 15)

===== COLUMN NAMES =====
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
      'alive', 'alone'],
      dtype='object')

===== DATA TYPES =====
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64

```

```

===== DATA TYPES =====
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      object
class         category
who           object
adult_male    bool
deck          category
embark_town   object
alive         object
alone         bool
dtype: object

===== DATASET INFORMATION =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64

```

```
===== STATISTICAL SUMMARY =====
      survived      pclass      age      sibsp      parch      fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean    0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std     0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min     0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%     0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%     0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%     1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max     1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```

```
===== MISSING VALUES =====
```

```
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
===== MISSING VALUES AFTER HANDLING =====
```

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck          0
embark_town   0
alive         0
alone         0
dtype: int64
```

```
===== SELECTED COLUMNS =====
```

```
      survived      sex      age      fare
0          0  male  22.000000    7.2500
1          1 female  38.000000   71.2833
2          1 female  26.000000    7.9250
3          1 female  35.000000   53.1000
4          0  male  35.000000    8.0500
5          0  male  29.699118    8.4583
6          0  male  54.000000   51.8625
7          0  male   2.000000   21.0750
8          1 female  27.000000   11.1333
9          1 female  14.000000   30.0708
```

```

===== FEMALE PASSENGERS =====
      sex   age   fare  survived
1  female  38.0  71.2833         1
2  female  26.0   7.9250         1
3  female  35.0  53.1000         1
8  female  27.0  11.1333         1
9  female  14.0  30.0708         1
10 female   4.0  16.7000         1
11 female  58.0  26.5500         1
14 female  14.0   7.8542         0
15 female  55.0  16.0000         1
18 female  31.0  18.0000         0

===== PASSENGERS WITH FARE > 100 =====
      sex   age   fare  survived
27  male   19.000000  263.0000         0
31  female 29.699118  146.5208         1
88  female 23.000000  263.0000         1
118 male   24.000000  247.5208         0
195 female 58.000000  146.5208         1
215 female 31.000000  113.2750         1
258 female 35.000000  512.3292         1
268 female 58.000000  153.4625         1
269 female 35.000000  135.6333         1
297 female  2.000000  151.5500         0

===== FEMALE PASSENGERS WITH AGE > 30 =====
      sex   age   fare  survived

```

```

===== FEMALE PASSENGERS WITH AGE > 30 =====
      sex   age   fare  survived
1  female  38.0  71.2833         1
3  female  35.0  53.1000         1
11 female  58.0  26.5500         1
15 female  55.0  16.0000         1
18 female  31.0  18.0000         0
25 female  38.0  31.3875         1
40 female  40.0   9.4750         0
52 female  49.0  76.7292         1
61 female  38.0  80.0000         1
85 female  33.0  15.8500         1

===== HIGHEST FARES =====
      sex   age   fare  survived
258 female  35.0  512.3292         1
737  male   35.0  512.3292         1
679  male   36.0  512.3292         1
88  female  23.0  263.0000         1
27  male   19.0  263.0000         0
341 female  24.0  263.0000         1
438  male   64.0  263.0000         0
311 female  18.0  262.3750         1
742 female  21.0  262.3750         1
118  male   24.0  247.5208         0

```

```

===== LOWEST FARES =====
      sex      age  fare  survived
271  male  25.000000   0.0         1
597  male  49.000000   0.0         0
302  male  19.000000   0.0         0
633  male  29.699118   0.0         0
277  male  29.699118   0.0         0
413  male  29.699118   0.0         0
674  male  29.699118   0.0         0
263  male  40.000000   0.0         0
466  male  29.699118   0.0         0
732  male  29.699118   0.0         0

===== SURVIVAL RATE BY GENDER =====
sex
female    0.742038
male      0.188908
Name: survived, dtype: float64

===== SURVIVAL RATE BY CLASS =====
class
First     0.629630
Second    0.472826
Third     0.242363
Name: survived, dtype: float64

```

```

===== FARE ANALYSIS BY CLASS =====
      sum      mean  min      max  count
class
First  18177.4125  84.154687  0.0  512.3292   216
Second  3801.8417  20.662183  0.0   73.5000   184
Third   6714.6951  13.675550  0.0   69.5500   491

===== PIVOT TABLE =====
sex      female      male
class
First    0.968085    0.368852
Second   0.921053    0.157407
Third    0.500000    0.135447

===== NUMPY ANALYSIS =====
Average Age: 29.7
Average Fare: 32.2
Maximum Fare: 512.3292
Minimum Fare: 0.0

===== USEFUL INSIGHTS =====
Overall Survival Rate: 38.38 %
Gender with Higher Survival Rate: female
Class with Highest Survival Rate: First
Average Ticket Fare: 32.2

```

Average Ticket Fare: 32.2

===== EXPORT =====

Cleaned dataset saved as: cleaned_titanic.csv

===== EXPORTED DATA =====

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	Unknown	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	Unknown	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	Unknown	Southampton	no	True

Exported Dataset Shape:
(891, 15)

Exported Dataset Columns:

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
      'alive', 'alone'],  
      dtype='object')
```

❖ Code link

https://drive.google.com/drive/folders/11LvWZ9mv0jdUYgRZ3eNMIq3ZF5b0bmRQ?usp=drive_link